

MILESTONE 2 REPORT: CORE SEARCH ENGINE IMPLEMENTATION

Course: SEG301 – Search Engines & Information Retrieval

Project: Smart Price Comparison Bot

Team: phap-bot / SEG301_Project

1 Project Overview

Following the data acquisition phase in **Milestone 1** (over **1,028,125 products** collected from seven e-commerce platforms), **Milestone 2** implements the core engine of the system: high-volume indexing and accurate search ranking.

Key Objectives

- **Indexing:** Implement the SPIMI algorithm to handle 1M+ documents under limited RAM.
- **Ranking:** From-scratch implementation of the BM25 ranking algorithm.
- **Performance:** Achieve sub-0.5s search latency.
- **Vietnamese Optimization:** Handle noisy marketplace data and language normalization.

2 Indexing Architecture: SPIMI Algorithm

To process over one million records on a personal computer, we implemented the **Single-Pass In-Memory Indexing (SPIMI)** algorithm in:

`src/indexer/spimi.py`

2.1 Implementation Workflow

1. **Block-based Processing:** Data is read in chunks of 10,000 products to maintain a safe memory footprint.
2. **In-Memory Construction:** For each block, a local inverted index is built storing document IDs and term frequencies (TF).

3. **Binary Serialization:** After completion, each block is sorted alphabetically and serialized as a `.pk1` binary file, immediately freeing RAM.
4. **K-Way Merge:** A streaming merge using a **heap (priority queue)** combines blocks into a final index without loading all data simultaneously.

2.2 Indexing Statistics

- **Total Documents:** 1,028,125
- **Vocabulary Size:** ~314,042 unique terms
- **Indexing Time:** Approximately 5–10 minutes (hardware dependent)

3 Ranking System: BM25 Algorithm

The ranking module implemented in:

`src/ranking/bm25.py`

was developed from scratch to enable deep customization for Vietnamese e-commerce search.

3.1 Core Mathematical Formula

The system applies the standard BM25 relevance scoring function:

$$Score(D, Q) = \sum_{q \in Q} IDF(q) \cdot \frac{f(q, D)(k_1 + 1)}{f(q, D) + k_1 \left(1 - b + b \frac{|D|}{avgdl}\right)}$$

Where:

- **IDF (Inverse Document Frequency)** weights rare terms higher than common terms.
- **TF** $f(q, D)$: frequency of query term in a document.
- **Length Normalization:** prevents keyword stuffing using parameters b and $avgdl$.

3.2 Advanced Vietnamese Optimizations

Several domain-specific improvements were implemented:

- **Noise Word Penalty:** Detects accessories or used parts and demotes them unless explicitly queried.
- **Model vs Capacity Distinction:** Avoids confusion between device models (e.g., iPhone 16) and storage capacity (16GB).
- **Byte Offsets:** Metadata stores byte positions enabling fast retrieval using `f.seek()`.

4 Performance & Results

The system was validated using the CLI application:

```
compare_prices.py
```

- **Search Latency:** Average **0.05s – 0.2s** across 1M documents.
- **Accuracy:** High-precision Top-10 results with noise filtering.
- **Stability:** Zero memory errors during indexing due to SPIMI architecture.

5 Conclusion

Milestone 2 successfully fulfills all technical objectives. By implementing SPIMI and BM25 from scratch, the project establishes a scalable high-performance search foundation ready for AI/LLM integration in Milestone 3.

Prepared By: Team phap-bot